VolunteerQuest

Gamified Local Volunteer Platform

Technical Architecture & Stack Reference

1. Project Overview

VolunteerQuest is a gamified web platform that connects users with local volunteer opportunities, rewarding civic engagement through points, badges, leaderboards, and streaks. It features a fully integrated Google Calendar sync so volunteers can one-click add events to their personal calendar, and organizations can publish recurring opportunities with ease.

Core platform pillars:

- Volunteer opportunity discovery (location-based, category-filtered)
- Gamification engine (XP, badges, streaks, leaderboard)
- Organization dashboard (post opportunities, manage signups)
- Google Calendar sync (OAuth2 + Calendar API)
- Real-time notifications for event changes and reminders

2. High-Level Architecture

The platform uses a modern full-stack architecture centered on Next.js for the frontend and API layer, with a lightweight Node.js/Express backend for heavier compute tasks, and a PostgreSQL database managed through Prisma ORM. The stack is intentionally kept lean and approachable.

Layer	Technology	Role
Frontend	Next.js 14 (App Router)	UI, routing, SSR/SSG
Styling	Tailwind CSS + shadcn/ui	Component library & design system
Backend API	Next.js API Routes + tRPC	Type-safe API endpoints
Database	PostgreSQL (via Neon or Supabase)	Primary data store
ORM	Prisma	Schema management, queries, migrations

Layer	Technology	Role
Auth	NextAuth.js (Auth.js v5)	OAuth2 + session management
Calendar Sync	Google Calendar API v3	Event creation, updates, sync
Deployment	Vercel	Hosting, edge functions, CI/CD

3. Frontend

3.1 Next.js 14 (App Router)

Next.js 14 with the App Router is the foundation. It handles server-side rendering for SEO-critical pages (opportunity listings, public org profiles), static generation for marketing pages, and client-side interactivity for the gamification dashboard.

- **App Router:** Layouts, loading states, and error boundaries per route segment.
- **Server Components:** Fetch opportunity listings and leaderboard data at the server level — no client waterfall.
- **Server Actions:** Form submissions (RSVP, sign-ups) handled server-side with no separate API route needed.
- **Route Handlers:** For webhook endpoints (Google Calendar push notifications) and OAuth callbacks.

3.2 Tailwind CSS + shadcn/ui

shadcn/ui provides accessible, unstyled components (buttons, dialogs, cards, badges) that are copy-pasted into the codebase and styled with Tailwind. This gives full design control without fighting a library.

- **Recommended shadcn components:** Badge (XP/level display), Card (opportunity tiles), Dialog (RSVP modal), Progress (XP bar), Avatar (user profile), Tabs (dashboard views).

> 💡 *shadcn/ui is not an npm dependency — you copy components into your project, so bundle size stays minimal.*

3.3 State Management

Given the Next.js App Router architecture, most server state is handled via Server Components and Route Handlers. Client-side state is minimal:

- **Zustand:** Lightweight global state for UI preferences, notification count, gamification animations.

- **TanStack Query (React Query):** Client-side data fetching and caching for real-time leaderboard and user stats.

3.4 Maps & Location

Volunteer opportunities are inherently location-based. Two options depending on budget:

Option	Library	Cost	Notes
Recommended	Mapbox GL JS	Free tier generous	Beautiful maps, clustering support
Free Alternative	Leaflet + OpenStreetMap	Free	No API key needed, basic styling
Google Maps	Google Maps JS SDK	Pay per use	Most familiar UX but billing complexity

4. Backend & API

4.1 tRPC (Type-Safe API)

tRPC creates end-to-end type-safe APIs between the Next.js frontend and backend — no REST schema or GraphQL SDL to maintain. API procedures are defined once in TypeScript and consumed directly on the client with full autocompletion.

- **Routers to build:** opportunitiesRouter, usersRouter, gamificationRouter, calendarRouter, organizationsRouter.
- **Middleware:** Auth middleware on protected procedures, rate limiting on public search endpoints.

💡 *tRPC v11 integrates natively with Next.js App Router. Use the @trpc/server adapter for Next.js route handlers.*

4.2 Database — PostgreSQL + Prisma

PostgreSQL is the right choice for this platform: relational data (users → signups → opportunities), JSONB for flexible gamification metadata, and geospatial queries via PostGIS for location-based filtering.

Recommended hosting — choose one:

- **Neon:** Serverless Postgres with branching (great for dev/staging), generous free tier, Vercel integration.

- **Supabase:** Managed Postgres + built-in Auth + Realtime — can replace Auth.js if you want an all-in-one BaaS.
- **Railway:** Dead-simple Postgres hosting, good DX, affordable.

Core Prisma schema models:

- **User:** id, name, email, image, displayName, role (VOLUNTEER | ORG_ADMIN), xp, level, streak, city, lat, lng, showOnLocalLeaderboard, lastActiveAt
- **Organization:** id, name, description, location, verified
- **Opportunity:** id, title, description, orgId, date, location (lat/lng), category, spotsTotal, spotsFilled, gcalEventId
- **Signup:** id, userId, opportunityId, status (PENDING | CONFIRMED | COMPLETED | CANCELLED)
- **Badge:** id, name, icon, xpRequired, category
- **UserBadge:** userId, badgeId, earnedAt
- **Group:** id, name, description, city, aggregateXp, inviteCode, isPublic
- **GroupMember:** id, userId, groupId, role (MEMBER | ADMIN), joinedAt — @@unique([userId, groupId])

4.3 Authentication — NextAuth.js (Auth.js v5)

Auth.js v5 handles all authentication flows. It integrates tightly with Next.js App Router and supports Google OAuth2 — which is essential since you need Google tokens for Calendar API access.

- **Google OAuth Provider:** Primary sign-in. Request Calendar scope at login so tokens are available for sync without a second auth step.
- **Email (Magic Link):** Fallback for organizations that prefer not to use Google.
- **Scopes to request:** openid, email, profile, https://www.googleapis.com/auth/calendar.events

> 💡 *Store the Google access_token and refresh_token in the Account table via Prisma adapter. You will need the refresh token to re-authenticate background calendar sync jobs.*

5. Google Calendar Integration

5.1 Architecture Overview

The calendar sync has two directions: volunteer-pull (adding an opportunity to a user's personal calendar) and org-push (organizations managing events from the platform with changes reflected on subscribed calendars).

Flow	Description	API Used
Volunteer clicks "Add to Calendar"	Create event on user's primary calendar	calendar.events.insert
Org updates opportunity date/time	Patch existing event on all subscriber calendars	calendar.events.patch
Opportunity cancelled	Delete event from subscriber calendars	calendar.events.delete
Platform calendar (org-owned)	Org maintains a public Google Calendar for their events	calendar.calendars.insert
Push notifications	Google notifies platform of external event changes	calendar.events.watch

5.2 Required Google APIs & Credentials

Enable the following in Google Cloud Console:

- **Google Calendar API:** Core event management.
- **Google People API:** Optional — enrich user profile at sign-in.
- **OAuth 2.0 Credentials:** Web Application type. Set authorized redirect URIs to your NextAuth callback URL.

Required OAuth Scopes:

- https://www.googleapis.com/auth/calendar.events — create/edit/delete events on user's calendar
- https://www.googleapis.com/auth/calendar.readonly — list user's calendars for calendar picker
- https://www.googleapis.com/auth/calendar — full access (needed if org manages a platform calendar)

> 💡 *Request only the minimum scopes at login. If the user later enables org features, trigger an incremental auth flow to request additional scopes.*

5.3 Google Client Libraries

Use the official Google APIs Node.js client library (@google/googleapis) server-side only — never expose credentials to the client browser.

- **googleapis npm package:** Wraps the REST API with typed methods. Use google.calendar('v3').
- **Token refresh:** Use OAuth2Client from google-auth-library to automatically refresh expired access tokens using stored refresh tokens.

- **Webhook endpoint:** Create a /api/gcal/webhook Route Handler to receive push notifications from Google when events are modified externally.

6. Gamification Engine

6.1 Core Mechanics

The gamification layer should be modular and data-driven so you can add new badges and XP events without code changes. Store all rules in the database.

Mechanic	Description	Implementation
XP (Experience Points)	Earned for signups, completions, streaks	Atomic DB update on Signup.status → COMPLETED
Levels	XP thresholds unlock new level titles	Computed from xp column; level table in DB
Badges	Achievement milestones (e.g., "10 events")	Background job checks badge eligibility on XP events
Streaks	Consecutive weeks with ≥1 completed event	Cron job resets streak if gap detected
Local Leaderboard	Top volunteers by XP within a city or zip radius	Geo-filtered indexed DB query; Redis cache
Group Leaderboard	XP rankings within a volunteer group	Filtered by GroupMember.groupId; real-time capable
Inter-group Leaderboard	Aggregate group XP — groups compete against each other	Sum of member XP per group; cached score on Group model
Impact Stats	Hours volunteered, causes helped	Aggregated from Signup records

6.2 Groups & Leaderboard System

Groups are a first-class feature — they allow friend circles, coworkers, university clubs, or community organizations to form teams, compete on a shared leaderboard, and track their collective impact. There are three distinct leaderboard scopes that must each be independently queryable and cached.

Leaderboard Scopes

Scope	Who Sees It	Ranked By	Filter
Local (city/neighborhood)	Any logged-in user	Individual XP	Geo radius or city string on User.city
Within a group	Group members only	Individual XP	GroupMember.groupId = groupId
Between groups (same area)	Any user / group page	Aggregate group XP	Group.city + sum of member XP

New Prisma Models for Groups

```
model Group {
   id            String        @id @default(cuid())
   name          String
   description   String?
   city          String        // for inter-group geo filtering
   aggregateXp   Int           @default(0) // cached sum, updated on XP
   events
   inviteCode    String        @unique @default(cuid())
   isPublic      Boolean       @default(false)
   members       GroupMember[]
   createdAt     DateTime      @default(now())
}

model GroupMember {
   id        String          @id @default(cuid())
   userId    String
   groupId   String
   role      GroupRole       @default(MEMBER) // MEMBER | ADMIN
   joinedAt  DateTime        @default(now())
   user      User            @relation(fields: [userId], references: [id])
   group     Group           @relation(fields: [groupId], references: [id])
   @@unique([userId, groupId])
}
```

Caching Strategy

All three leaderboard scopes are read-heavy and can tolerate slight staleness (a 60-second refresh window is imperceptible to users). Cache all three in Vercel KV (Upstash Redis).

Leaderboard	Redis Key Pattern	TTL	Invalidation Trigger
Local	leaderboard:local:{city}	60s	Any user in that city earns XP
Within group	leaderboard:group:{groupId}	30s	Any member of the group earns XP
Between groups	leaderboard:inter:{city}	120s	Group.aggregateXp updated

> 💡 *On every XP award, fire an Inngest event xp.awarded that updates Group.aggregateXp atomically (Prisma $transaction) and deletes the relevant Redis cache keys. The next page request regenerates the cache from a fresh DB query.*

Local Leaderboard Query

The local leaderboard filters by city string on the User model (populated at signup from the Google profile or a manual input step). For finer-grained geo filtering by radius, add lat/lng to User and use a PostGIS ST_DWithin query.

```
// tRPC: leaderboardRouter.local
const local = await prisma.user.findMany({
  where: { city: input.city },
  orderBy: { xp: 'desc' },
  take: 50,
  select: { id, name, image, xp, level, streak }
});
```

Within-Group Leaderboard Query

```
// tRPC: leaderboardRouter.withinGroup
const members = await prisma.groupMember.findMany({
  where: { groupId: input.groupId },
  include: { user: { select: { id, name, image, xp, level, streak } } },
  orderBy: { user: { xp: 'desc' } },
});
```

Inter-Group Leaderboard Query

Groups compete using the cached aggregateXp column, which is incrementally updated via Inngest rather than recomputed on every request.

```
// tRPC: leaderboardRouter.betweenGroups
const groups = await prisma.group.findMany({
  where: { city: input.city },
  orderBy: { aggregateXp: 'desc' },
  take: 20,
  include: { _count: { select: { members: true } } }
});
```

Group Features & UX Flows

Feature	Description
Create group	Any user can create a group — they become ADMIN. Group gets a unique invite code and optional public listing.
Join via invite link	/join/{inviteCode} — one-click join, no approval required for private groups by default.
Join request (optional)	ADMIN can toggle approval-required. Pending joins stored as GroupMember with status PENDING.
Group profile page	Public page showing group name, member count, aggregate XP, top 3 members, and recent activity.
Multiple group membership	A user can belong to multiple groups. XP contribution counts toward all groups simultaneously.
Group badges	Milestone badges for the group as a whole (e.g., "500 hours collectively") stored on Group, not User.

Privacy Considerations

- **Opt-out of local leaderboard:** Add a showOnLocalLeaderboard boolean to the User model. Default true, but users can hide themselves without losing XP.
- **Group privacy:** isPublic controls whether the group appears in the inter-group leaderboard and is discoverable by non-members.
- **Display names:** Users can set a separate displayName distinct from their real name for public leaderboards.

6.3 Background Jobs

Badge awarding, streak calculation, and digest emails require background processing outside the request cycle.

- **Vercel Cron Jobs:** Built into Vercel — define cron schedules in vercel.json to call internal API routes. Good for streak resets and reminders.
- **Inngest:** Event-driven background functions with retries. Best for badge checks triggered by user actions (event-driven, not time-based).
- **Trigger.dev:** Alternative to Inngest; open-source friendly, self-hostable.

💡 *For an MVP, Vercel Cron + a simple queue table in Postgres is sufficient before adding a dedicated job runner.*

7. Notifications & Real-Time

7.1 Email Notifications

Transactional emails for signup confirmation, reminders, badge awards, and weekly digests.

Provider	Free Tier	Notes
Resend	3,000 emails/month	Best DX, React Email templates, recommended
SendGrid	100 emails/day	Mature platform, good deliverability
Postmark	100 emails/month	Excellent deliverability for transactional mail

- **React Email:** Build email templates as React components, render to HTML server-side. Works perfectly with Resend.

7.2 In-App Real-Time Updates

For live leaderboard updates and instant badge notifications:

- **Pusher Channels / Ably:** Managed WebSocket service. Simple to integrate, generous free tier.
- **Supabase Realtime:** If you choose Supabase as your database, Realtime is built in — listen to Postgres changes directly.
- **Server-Sent Events (SSE):** Lightweight alternative for one-way push (leaderboard updates, badge popups). No external service needed.

8. Deployment & Infrastructure

8.1 Vercel (Recommended)

Vercel is the natural hosting target for Next.js — zero-config deployment, edge network, preview deployments per branch, and built-in analytics.

- Automatic HTTPS and custom domain support
- Environment variable management per deployment environment
- Edge Middleware for auth-protected routes
- Vercel Cron for background job scheduling

- Vercel KV (Redis-compatible) for leaderboard caching and session storage

8.2 Environment Variables Checklist

Required secrets to configure before launch:

Variable	Purpose
DATABASE_URL	PostgreSQL connection string (Neon/Supabase/Railway)
NEXTAUTH_SECRET	Auth.js JWT signing key
NEXTAUTH_URL	Public base URL of the app
GOOGLE_CLIENT_ID	Google OAuth app client ID
GOOGLE_CLIENT_SECRET	Google OAuth app client secret
GOOGLE_CALENDAR_WEBHOOK_SECRET	Verify incoming Calendar push notifications
RESEND_API_KEY	Transactional email sending
INNGEST_EVENT_KEY	Background job authentication (if using Inngest)
MAPBOX_TOKEN	Map display on opportunity listings page
NEXT_PUBLIC_APP_URL	Client-accessible public URL

9. Analytics & Monitoring

9.1 Product Analytics

- **Vercel Analytics:** Built-in, privacy-friendly page view and performance tracking.
- **PostHog:** Open-source product analytics with session recording, funnels, and feature flags. Self-hostable or cloud. Highly recommended for tracking gamification engagement.

9.2 Error Monitoring

- **Sentry:** Industry-standard error tracking. Next.js SDK captures both server and client errors. Free tier covers most early-stage needs.

9.3 Logging

- **Axiom:** Structured logging with Vercel integration. Low cost, powerful query interface.
- **Logtail / Better Stack:** Alternative — good free tier, easy Vercel setup.

10. AI Feature: Impact Storytelling

Impact Storytelling is the highest-ROI AI feature to build first. It requires no new infrastructure beyond the Anthropic API, triggers off an event already in your system (Signup status → COMPLETED), and transforms generic gamification points into emotionally resonant personal narratives that drive long-term retention.

Instead of showing a volunteer "+50 XP earned," the platform generates a message like:

> *"This month you contributed 8 hours to food security in your city. Combined with your previous volunteering, you've helped provide an estimated 340 meals to families in Queens — and you're in the top 12% of volunteers in your area."*

This is the feature that turns a one-time volunteer into a habitual one, and creates content users want to share.

10.1 Data Flow

Step	Action	Where
1	Signup.status updated to COMPLETED	tRPC mutation / Server Action
2	Inngest event fired: volunteer.completed	Background job trigger
3	User history pulled from DB (all signups, XP, badges, causes)	Prisma query
4	Prompt assembled with user context + org/event metadata	Inngest function
5	Claude API call → narrative generated	Anthropic SDK (server-side)
6	Narrative stored on ImpactStory model	Prisma write
7	Displayed on profile dashboard + emailed via Resend	Next.js UI + Resend

10.2 New Prisma Model

Add an ImpactStory model to your schema:

```
model ImpactStory {
```

```
    id          String   @id @default(cuid())
    userId      String
    signupId    String   @unique
    narrative   String   // AI-generated story
    hoursTotal  Float    // cumulative at time of generation
    percentile  Int?     // city leaderboard rank at generation time
    createdAt   DateTime @default(now())
}
```

10.3 Claude Prompt Template

The prompt is assembled server-side in the Inngest function and should never be exposed to the client. Key principles: ground every claim in real data, use second-person voice, and keep it under 4 sentences so it's digestible on mobile.

```
System: You are an impact writer for a volunteer platform. Write warm, specific, data-grounded impact narratives in second person. Never fabricate statistics. Keep responses to 3-4 sentences. Do not use generic phrases like "making a difference."

User: Volunteer: {name}. Just completed: {event_title} with {org_name} ({event_duration}h). Cause category: {category}. Total volunteer history: {total_events} events, {total_hours}h across {causes} causes. XP this month: {xp}. City leaderboard percentile: {percentile}. Write their impact story.
```

💡 *Inject all template variables from the DB query result before sending to the API. Store the final narrative verbatim — never regenerate on page load.*

10.4 Required Packages

Package	Purpose
@anthropic-ai/sdk	Official Anthropic TypeScript SDK for Claude API calls
inngest	Event-driven background function that triggers on volunteer.completed
react-email + resend	Render impact story as HTML email and deliver post-completion

11. AI Feature: Semantic Volunteer–Opportunity Matching (RAG)

Keyword search fails volunteers who don't know the exact terms orgs use. Semantic matching uses vector embeddings to understand intent, surfacing opportunities that are conceptually relevant even when no words overlap. A volunteer who says "I love animals" will be matched to "Shelter Dog Socialization Program" without the word "animals" appearing in the listing.

This is implemented as a Retrieval-Augmented Generation (RAG) pipeline: volunteer queries and opportunity descriptions are converted to embedding vectors, stored in pgvector (a Postgres extension), and retrieved via cosine similarity — keeping everything in your existing database with no new vector database service required.

11.1 Architecture

Stage	Description	Tool
Embed opportunities	On creation/update, embed the opportunity title + description + category	OpenAI text-embedding-3-small
Store vectors	Save embedding vector alongside the opportunity row	pgvector (Postgres extension)
Embed user query	At search time, embed the user's query or profile summary	OpenAI text-embedding-3-small
Similarity search	Find top-K opportunities by cosine similarity to query vector	pgvector <=> operator via Prisma raw SQL
Re-rank results	Apply distance, date, spots-available boosts to raw similarity scores	Server-side scoring function
Optional: RAG chat	Pass top results as context to Claude for conversational discovery UI	Anthropic SDK

11.2 pgvector Setup

pgvector is a Postgres extension — enable it once and it works natively with Neon and Supabase. No separate vector database needed.

```
-- Run once in your database migration:
CREATE EXTENSION IF NOT EXISTS vector;

-- Add embedding column to Opportunity:
ALTER TABLE "Opportunity" ADD COLUMN embedding vector(1536);

CREATE INDEX ON "Opportunity" USING ivfflat (embedding vector_cosine_ops);
```

💡 *1536 dimensions matches OpenAI text-embedding-3-small output. The ivfflat index makes similarity queries fast as your opportunity table grows.*

11.3 Embedding Pipeline

Two embedding jobs are required: a backfill for existing opportunities and an on-create trigger for new ones.

- **On opportunity creation/update:** Fire an Inngest event opportunity.upserted. The function calls the OpenAI Embeddings API with the concatenated title + description + category string, then writes the returned vector to the embedding column via Prisma raw SQL.
- **Backfill job:** A one-time Vercel Cron or manual script that iterates all existing opportunities without embeddings and generates them in batches of 100 (respect OpenAI rate limits).
- **User profile embedding:** On signup completion or profile update, embed a summary string built from the user's stated interests, past opportunity categories, and bio. Store on the User model as profileEmbedding vector(1536).

11.4 Similarity Search Query

The discovery page calls a tRPC query that runs a raw SQL similarity search via Prisma, then applies server-side re-ranking before returning results to the client.

```
// In your opportunitiesRouter (tRPC)
const results = await prisma.$queryRaw`
  SELECT id, title, date, location, spots_filled, spots_total,
         1 - (embedding <=> ${queryVector}::vector) AS similarity
  FROM "Opportunity"
  WHERE date > NOW() AND spots_filled < spots_total
  ORDER BY embedding <=> ${queryVector}::vector
  LIMIT 20`
```

💡 *The <=> operator is pgvector's cosine distance operator. Subtract from 1 to get cosine similarity (1 = identical, 0 = unrelated). Filter out full or past events before similarity search for efficiency.*

11.5 Conversational Discovery (Optional RAG Layer)

Once the similarity search pipeline is working, adding a chat interface is straightforward. The retrieved opportunities become the context window for a Claude completion that responds conversationally to the volunteer's query.

- **Input:** User natural language query (e.g., "outdoor activities on weekends that help kids")

- **Step 1:** Embed the query → run pgvector similarity search → retrieve top 5–8 matching opportunities
- **Step 2:** Format retrieved opportunities as structured context (title, org, date, description, spots)
- **Step 3:** Send to Claude with a system prompt instructing it to recommend from the provided list only — preventing hallucination of non-existent events
- **Step 4:** Stream Claude's response to the UI using Vercel AI SDK's useChat hook

> 💡 *The "from the provided list only" constraint in the system prompt is critical. Without it, the model may describe opportunities that don't exist on your platform.*

11.6 Required Packages

Package	Purpose
openai	OpenAI SDK — used only for the Embeddings API (text-embedding-3-small)
@anthropic-ai/sdk	Claude API for conversational RAG layer (if building chat UI)
ai (Vercel AI SDK)	useChat hook + streamText for streaming Claude responses in Next.js
pgvector	Postgres extension — enabled via migration, no npm package needed
inngest	Triggers embedding jobs on opportunity.upserted events

11.7 Cost Estimate

Embeddings are extremely cheap. At scale this feature costs pennies.

Operation	Model	Cost Estimate
Embed 1,000 opportunities	text-embedding-3-small	~$0.002 total (one-time backfill)
Embed 10,000 user queries/day	text-embedding-3-small	~$0.02/day
RAG Claude response (1K tokens)	claude-haiku-3-5	~$0.001 per conversation turn
Impact story generation	claude-sonnet-4-5	~$0.003 per story

12. Complete Stack Reference

Category	Tool / Service	Tier
Framework	Next.js 14 (App Router)	Core
Language	TypeScript	Core
Styling	Tailwind CSS	Core
Components	shadcn/ui	Core
API Layer	tRPC v11	Core
ORM	Prisma	Core
Database	PostgreSQL (Neon)	Core
Auth	Auth.js v5 (NextAuth)	Core
Google Calendar	googleapis npm + Google Cloud Console	Core
Maps	Mapbox GL JS	Core
Email	Resend + React Email	Core
State (client)	Zustand + TanStack Query	Core
Deployment	Vercel	Core
Background Jobs	Vercel Cron + Inngest	Core
Caching / KV	Vercel KV (Upstash Redis)	Core
Real-time	Pusher Channels	Optional
Analytics	PostHog	Optional
Error Tracking	Sentry	Recommended
Logging	Axiom	Recommended
AI: Impact Stories	Anthropic Claude (claude-sonnet-4-5) + Inngest	Core AI
AI: Embeddings	OpenAI text-embedding-3-small + pgvector	Core AI
AI: RAG Chat	Claude + Vercel AI SDK (useChat)	Core AI

13. Suggested Build Phases

Phase 1 — MVP (Weeks 1–4)

- Next.js project scaffold with Tailwind + shadcn/ui
- PostgreSQL + Prisma schema (User, Opportunity, Signup, Organization)
- Auth.js with Google OAuth (request Calendar scope)
- Opportunity listing page with basic map view
- RSVP / signup flow
- Basic XP award on signup completion
- "Add to Google Calendar" button (single-event insert)

Phase 2 — Gamification + AI (Weeks 5–7)

- Full XP + level system with progress UI
- Badge definitions and award engine
- Leaderboard (global + city-filtered)
- Streak tracking via Vercel Cron
- Email notifications via Resend (confirmation, reminders)
- AI Impact Storytelling: Inngest job + Claude API + ImpactStory model
- pgvector extension enabled + opportunity embedding pipeline

Phase 3 — Calendar Sync, Orgs & RAG (Weeks 8–10)

- Organization dashboard (create/edit/cancel opportunities)
- Bi-directional Calendar sync (patch/delete on org updates)
- Google Calendar push notifications webhook
- In-app real-time badge + leaderboard updates
- Semantic similarity search on opportunity discovery page
- Conversational RAG chat UI (Vercel AI SDK + Claude + pgvector)
- PostHog analytics instrumentation